



Junta de Andalucía
Consejería de Educación y Deporte

UNIDAD DE TRABAJO 11

INTERFACES GRÁFICAS

—

Módulo de Programación



This work is licensed under a [Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-nc-sa/4.0/).

Autor: Fran Gómez

RA5 Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases

- f) Se han utilizado las herramientas del entorno de desarrollo para crear interfaces gráficos de usuario simples.
- g) Se han programado controladores de eventos.
- h) Se han escrito programas que utilicen interfaces gráficos para la entrada y salida de información.

1. Introducción
2. Librerías para realizar interfaces gráficas
 - a. ¿AWT, SWING, FX?
3. Componentes
 - a. Contenedores
 - b. Controladores
4. Eventos
5. Layout
6. Menús

Character User Interfaces vs Graphical User Interface



Aplicaciones gráficas realizadas con Java



← → ↻ 🏠 localhost:8080/users/1

Spring React CRUD Users About frodo@local ▾

Update User

Email
frodo@local

Password
Password (leave blank)

First name
Frodo

Notes
Ring bearer

Users

First Name	Last Name	Email		
Frodo	Baggins	frodo@local	Ring bearer	Edit Delete
Bilbo	Baggins	bilbo@local	Burglar	Edit Delete
Gandalf	the Grey	gf@local	Wizard	Edit Delete
Legolas	Greenleaf	lego@local	Elf prince	Edit Delete
Sam	Gamgee	sam@local	The gardener	Edit Delete

« > »

Please sign in

Username

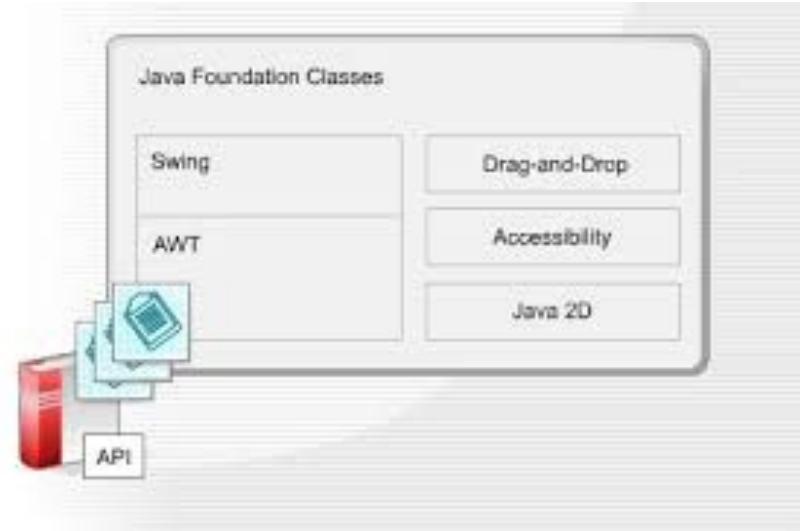
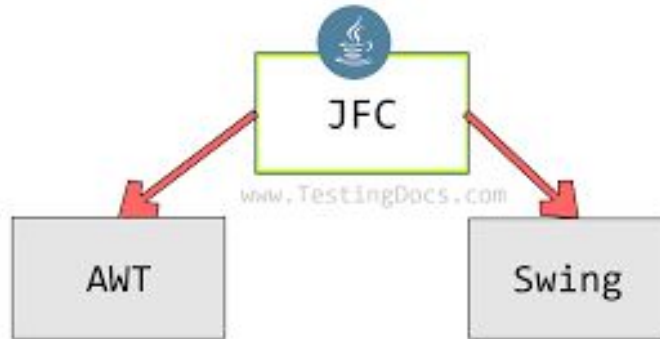
Password

Librerías para GUI en Java



1º DAW
Programación

UD 11: AWT



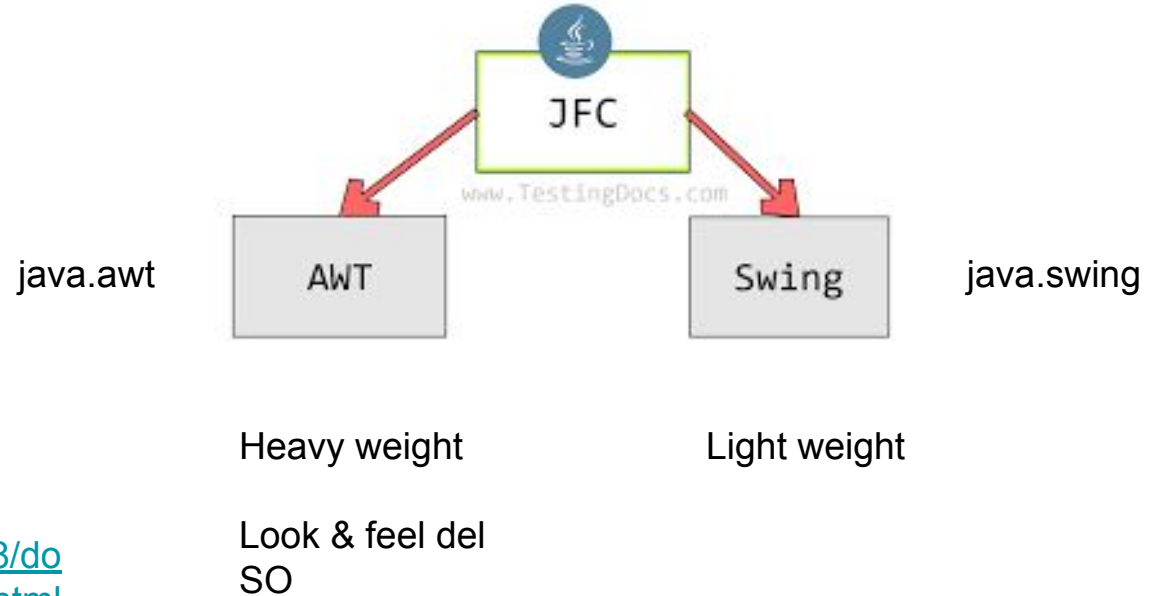
Librerías para GUI en Java: Conceptos previos



1º DAW
Programación

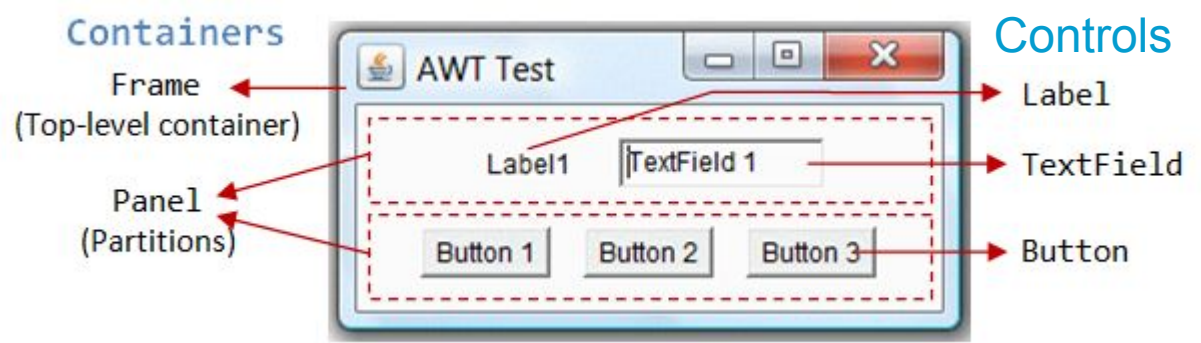
UD 11: AWT

- Framework
- Librería
- Paquete
- Componente



<https://docs.oracle.com/javase/8/docs/api/java/awt/package-frame.html>

Componentes AWT



Componentes

- Contenedores: contienen otros componentes

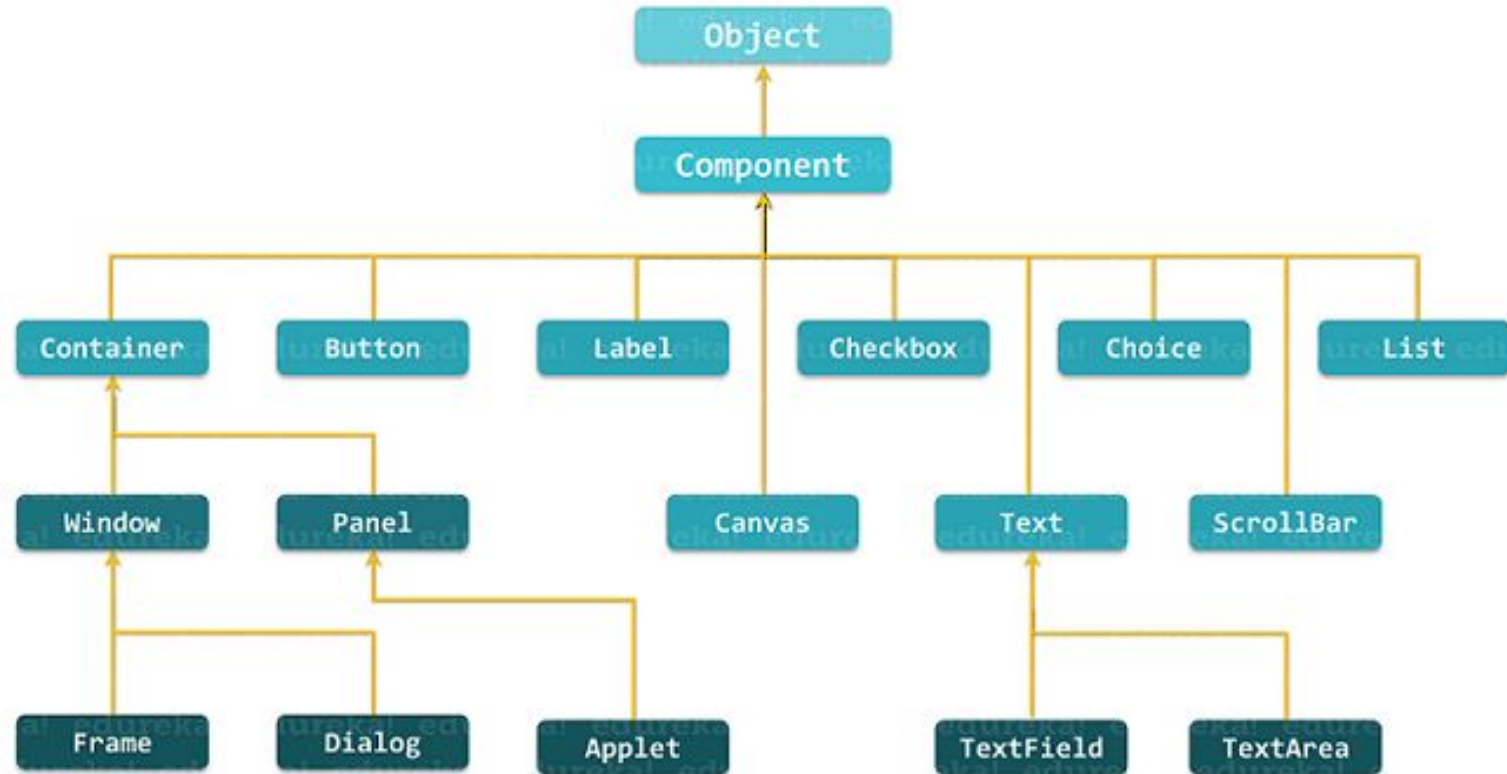
Window, Frame, Pane...

- Controles: realizan alguna acción

Button, TextField, Label...

Todos los elementos que se pueden ver se llaman componentes

Jerarquía de los componentes



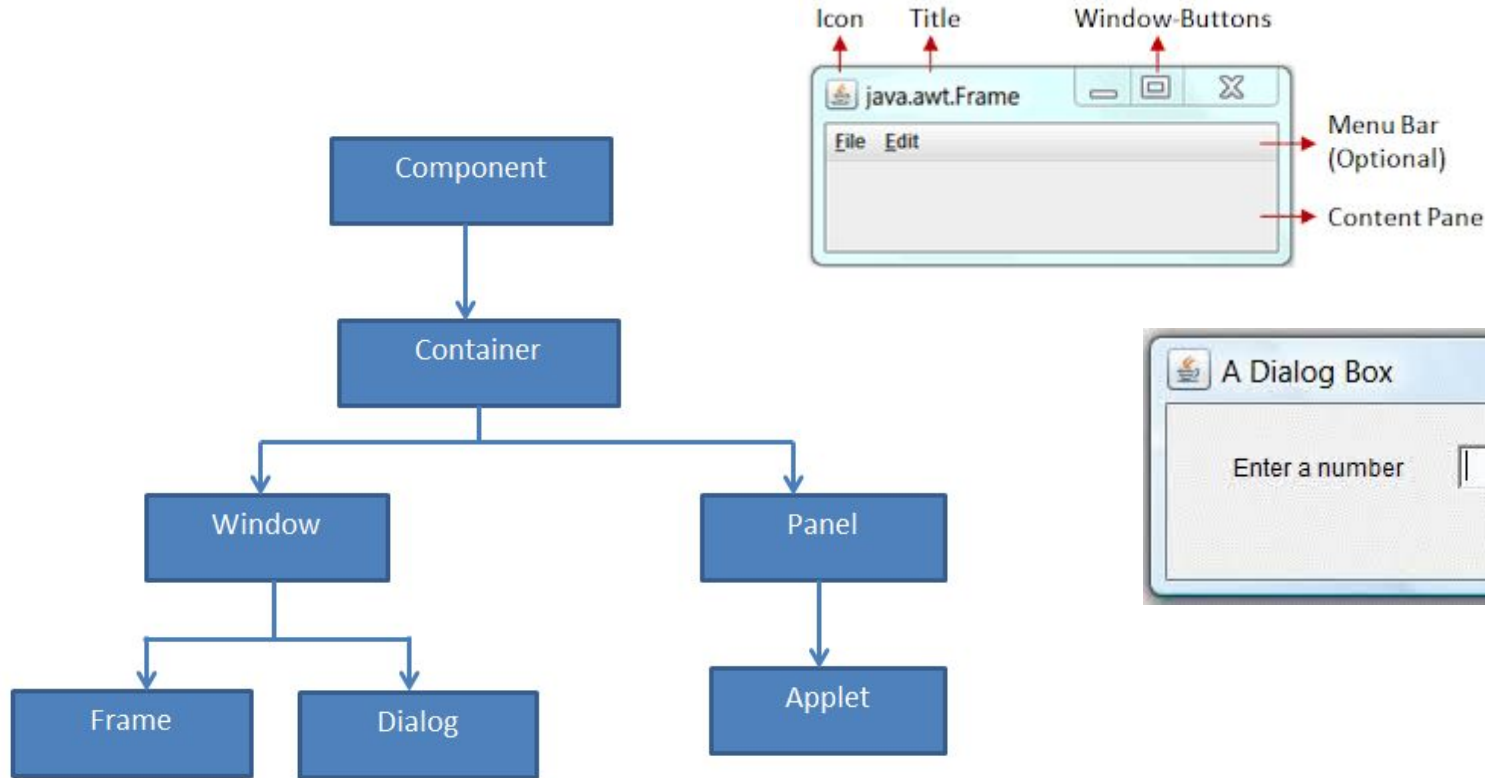
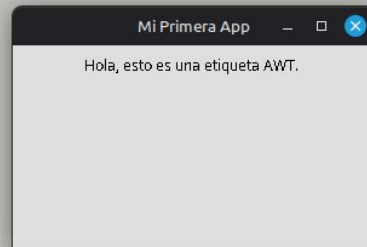


Fig : AWT Hierarchy



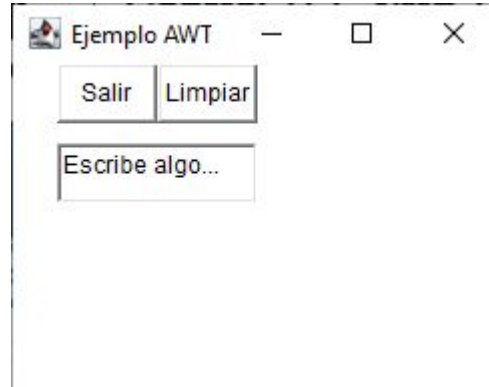
```
3 import java.awt.FlowLayout;
4 import java.awt.Frame;
5 import java.awt.Label;
6
7 public class MiPrimeraVentana {
8
9     public static void main(String[] args) {
10         // 1. CREAR EL CONTENEDOR (La ventana vacía)
11         Frame ventana = new Frame("Mi Primera App");
12         ventana.setSize(300, 200); // Ancho: 300 píxeles, Alto: 200 píxeles
13         ventana.setLayout(new FlowLayout()); // El "organizador" automático
14
15         // 2. CREAR EL CONTROL (La etiqueta de texto)
16         Label etiqueta = new Label("Hola, esto es una etiqueta AWT.");
17
18         // 3. METER EL CONTROL DENTRO DEL CONTENEDOR
19         ventana.add(etiqueta);
20
21         // 4. HACERLA VISIBLE (Por defecto nacen invisibles en memoria)
22         ventana.setVisible(true);
23     }
24 }
25 }
```



Ejercicio 1



Implementa una ventana como la siguiente



Layout (Posicionamiento de los componentes)



1º DAW
Programación

UD 11: AWT

Manual:

`setLayout(null)` → Posicionamiento absoluto

`setBounds(X, Y, Ancho, Alto)`

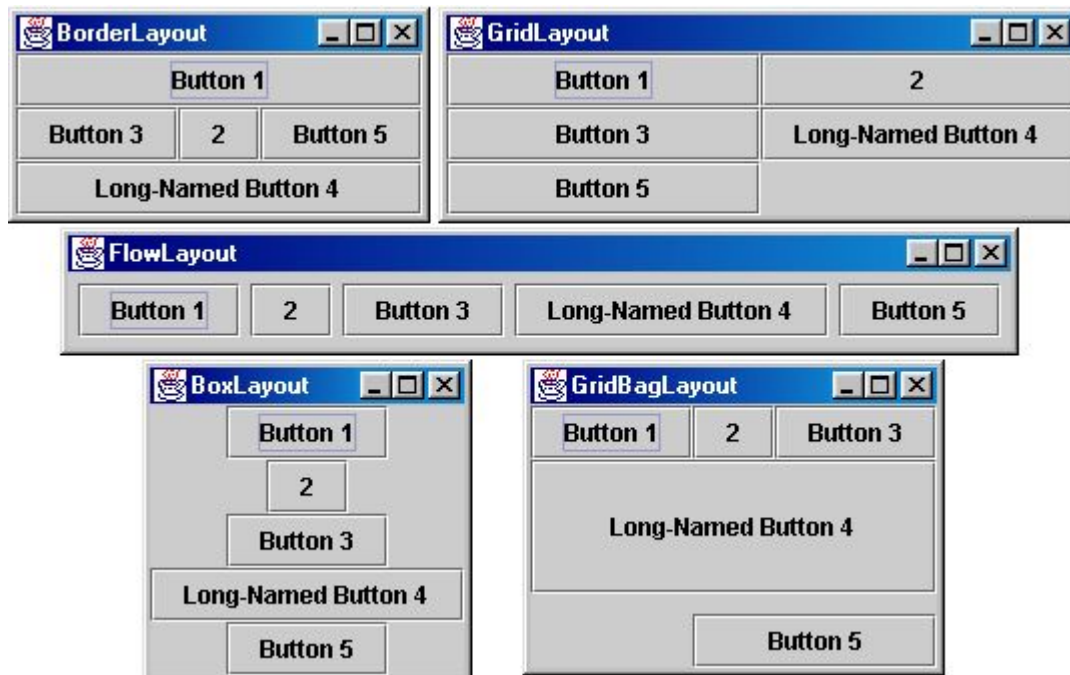
Automático:

`setLayout(new FlowLayout())`

Interface **LayoutManager**

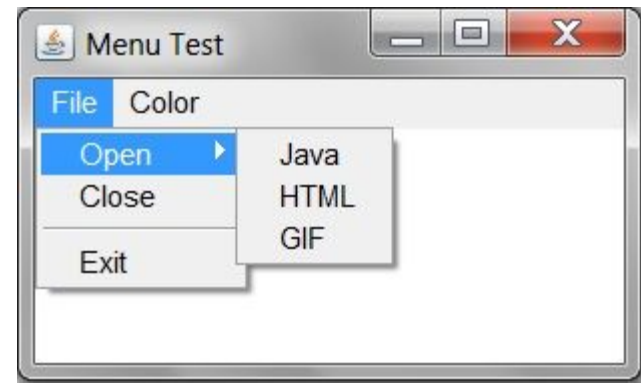
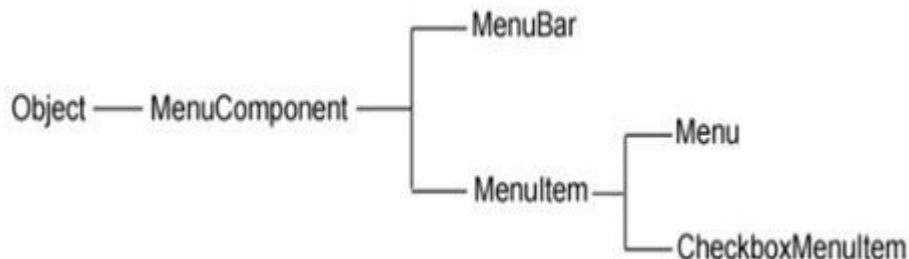
Implementaciones:

- BorderLayout
- CardLayout
- FlowLayout
- GridLayout



1. **MenuBar (La Barra de Menú):** Es la tira horizontal invisible que va arriba del todo de la ventana. Solo puede haber una por ventana.
2. **Menu (El Desplegable):** Son las secciones principales que cuelgan de la barra (ej: "Archivo", "Editar", "Ayuda"). Al hacer clic en ellas, despliegan una lista.
3. **MenuItem (La Opción/El Botón):** Son los elementos finales clicables dentro de cada menú (ej: "Nuevo", "Guardar", "Salir"). **Estos son los que disparan los eventos.**

💡 **Nota:** Un **Menu** también puede contener otro **Menu** dentro (creando un submenú desplegable).



Menús

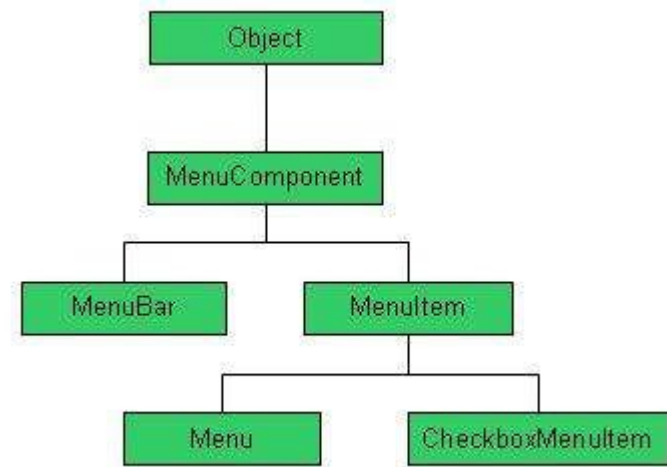


Para hacer el menú, hay que ir creando los elementos de la jerarquía:

```
barra = new MenuBar();    // 1º La Barra Principal  
menu = new Menu("File"); // 2º El Desplegable  
item = new MenuItem("Exit"); // 3º Las Opciones finales
```

y ensamblarlos abajo hacia arriba:

```
menu.add(item);  
barra.add(menu);  
frame.setMenuBar(barra);
```



¿Por qué no se cerraba la ventana?

Suceso que ocurre durante la ejecución del programa (como hacer clic en un botón, escribir en un campo o cerrar una ventana) y que el programa puede detectar y gestionar mediante código.

Nos van a permitir que la interfaz gráfica responda a las acciones del usuario.

Cuando el usuario realiza una acción, automáticamente se genera un objeto con información sobre esta acción.

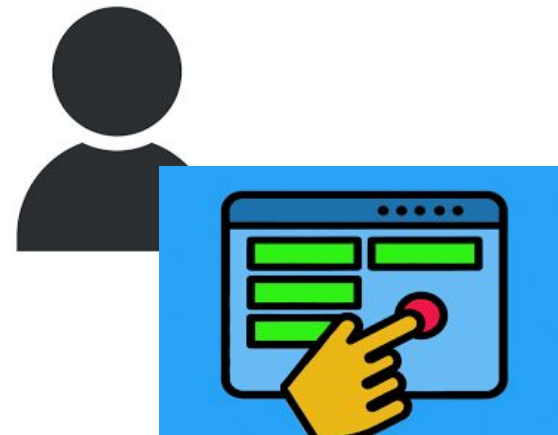


¿Quién los genera?

- Puede ser generado por el usuario (clic, teclado)
- Por el sistema (temporizador, cambio de estado)
- Por otro componente del programa

Ejemplos:

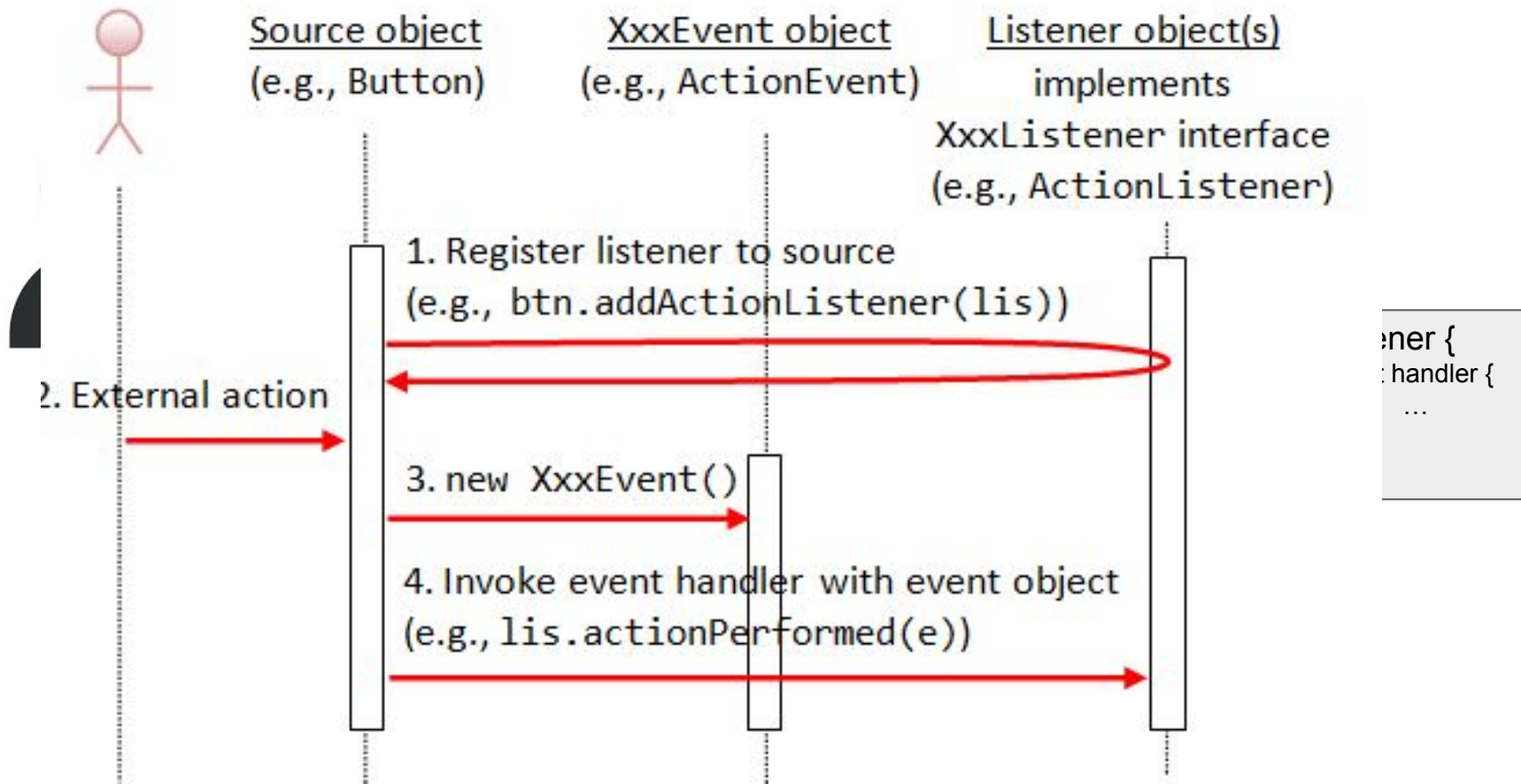
- El usuario hace clic en un botón → `ActionEvent`
- El cursor entra en un área de la ventana → `MouseEvent`
- Se cierra una ventana → `WindowEvent`
- El usuario presiona una tecla → `KeyEvent`



Un evento tiene 3 partes implicadas:

1. **Una fuente** ➤ El componente que genera el evento (por ejemplo, un botón).
2. **Un objeto evento** ➤ Contiene información sobre el suceso ocurrido.
3. **Un oyente (listener)** ➤ El código que se activa cuando ocurre el evento.

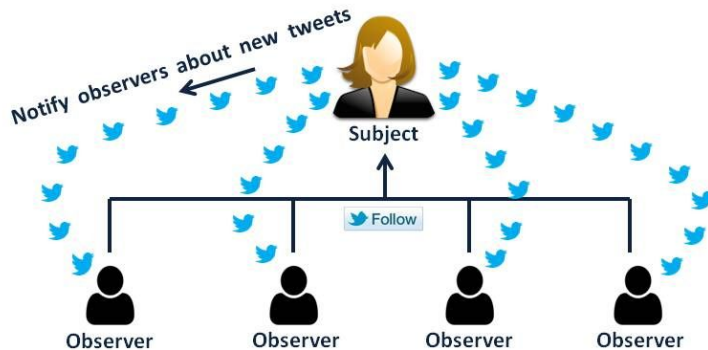
Eventos: Funcionamiento



Los listeners que queramos se encarguen de manejar un evento, debemos registrarlos en el componente para que sean notificados por éste cuando se genere el evento → Observer pattern.

Es como subscribirse a un canal de noticias

Observer Design Pattern



Evento: Registrar listeners



Para registrar un listener en un componente usamos el método:

```
someComponent.addXXXListener(instanceOfXXXListener);
```

Para dar de baja el listener:

```
someComponent.removeXXXListener(instanceOfXXXListener);
```

donde XXX es la clase de evento: Action, Component, Key, Window...

<https://docs.oracle.com/javase/tutorial/uiswing/events/actionlistener.html>

Eventos: Ejemplo



Frame v

ventana

ventana

Button

ventana

```
public class MiListener implements ActionListener {  
  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        System.out.println("Botón pulsado!");  
    }  
}
```

```
ActionListener miListener = new MiListener();
```

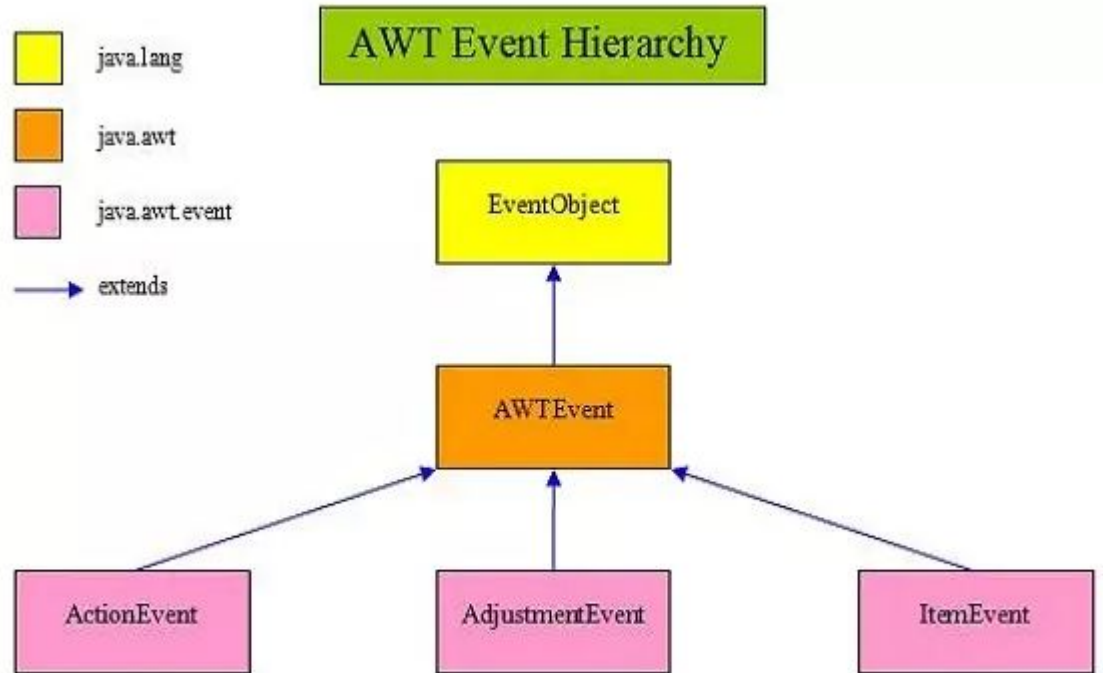
```
miBoton.addActionListener(miListener);
```

```
ventana.setVisible(true);
```

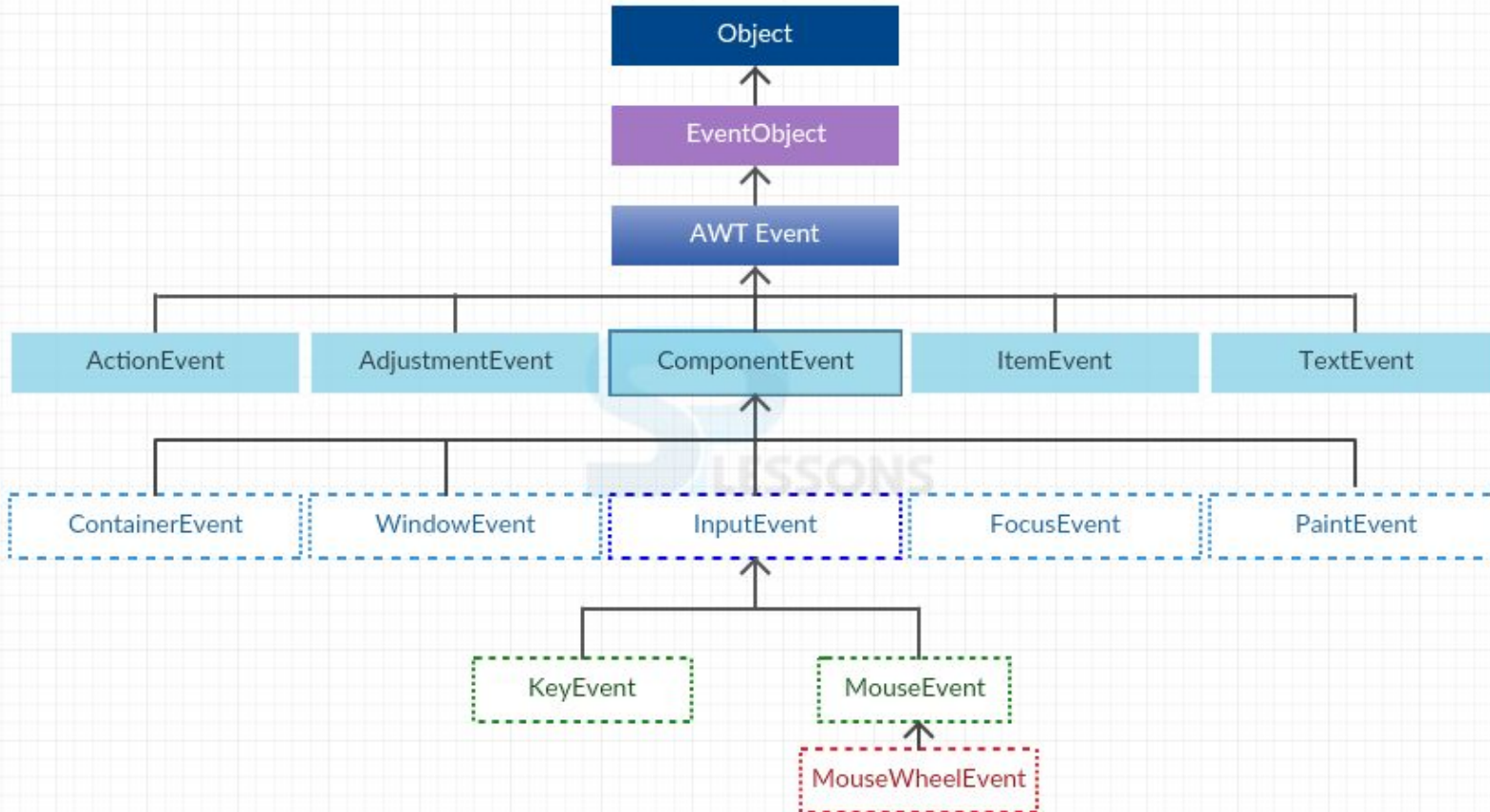

Ejemplo

```
Button button = new Button("Salir");  
button.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        ((Frame)button.getParent()).dispose();  
    }  
});
```

Eventos: Clases para los Eventos



Eventos: Clases para los Eventos



Eventos: Clases e Interfaces



Event class	Acciones sobre Componentes	Listeners interfaces	Adapter
ActionEvent	Presionar un Button, escribir en un TextField, doble click en una List, seleccionar un MenuItem	ActionListener <code>actionPerformed(ActionEvent e)</code>	-
WindowEvent	Cualquier acción sobre el Component Window	WindowListener	WindowAdapter
MouseEvent	Mover, arrastrar, hacer click... sobre un Component	MouseListener MouseMotionListener	MouseAdapter
...			

Ejemplo usando herencia e implementación



```
import javax.swing.*.*;
import java.awt.event.*;

public class EjemploEvento extends JFrame implements ActionListener {
    JButton boton;

    public EjemploEvento() {
        boton = new JButton("Haz clic");
        boton.addActionListener(this); // Registrar el oyente
        add(boton);
        setSize(200, 200);
        setLayout(null);
        boton.setBounds(50, 80, 100, 30);
        setVisible(true);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(this, "¡Botón pulsado!");
    }

    public static void main(String[] args) {
        new EjemploEvento();
    }
}
```

- Swing
- JavaFX

Actividades finales



1º DAW
Programación

UD 11: AWT

1. Práctica
2. Portfolio
3. Feedback